

Codex 为什么会走出代码

Tibo x OpenAI Forum：Why Codex Matters Beyond Code

官方直播回放 / 官方逐字稿 • 约 43 分钟 • 会谈日期 2026.05.13

一句话结论

这场是 Tibo 从“Coding Agent”转向“通用个人 Agent”最完整的一次公开解释：非代码任务已经成为主流使用，Agent 的下一步不是更多聊天能力，而是长期目标、后台执行、跨应用 context 和企业级 trust。

四个最值得关注的信号

01 非代码主流 Tibo 表示 Codex 的多数任务已经不是 coding；工程师本来也只有一部分时间在写代码。	02 Chief of Staff 他自己每天发出上百个小任务，让 Agent 管理信息、排期、计算资源与风险。
03 Goal -> 24/7 从 turn-based task 走向长目标，再走向持续运行的 Agent。	04 企业瓶颈是 Trust 企业 adoption 首要障碍不是能力，而是安全、权限、数据外泄和可控性。

说明：下文将“访谈观点”“技术解读”“方向推演”分开呈现；预测不视为正式产品承诺。

OVERVIEW

访谈背景与章节地图

受访者：Thibault “Tibo” Sottiaux，OpenAI Core Product / Codex 相关负责人。

对谈方：Chris Nicholson（OpenAI Global Affairs）

形式：官方直播回放 / 官方逐字稿；时长：约 43 分钟

访谈观点

技术解读

方向推演

外部核验

章节地图 / 讨论顺序

时间 / 顺序	主题
01:31	早期 Codex Web：云端看 Repo、开 PR
02:39	为什么因配置摩擦与模型可靠性转向本地
03:36	非代码任务开始爆发
06:06	发布期多 Agent 协调让 Tibo 意识到“不只是软件工程”
17:52	Tibo 每天 >100 个 Codex 小任务
24:11	/goal：小时、天、周级任务
26:04	未来 24/7 持续 Agent
30:32	企业 adoption：Trust、Sandbox、Auto-review

阅读方法

先看 Tibo 明确表达的系统设计原则，再看这些原则对 Agent Runtime、Serving、Sandbox、Memory、Multi-agent 或软件工程流程的含义。不要把方向判断直接当作已经落地的功能。

CORE IDEAS

1. 从 Coding Agent 到 “注意力操作系统”

- 关键转折不是 Codex 突然学会写报告，而是为了把软件工程任务做完整，它本来就必须接入 ticket、文档、Slack、bug、on-call、架构讨论等非代码 context。能力一旦跨过可靠性阈值，使用自然外溢到一般知识工作。
- Tibo 自己把 Agent 当 “chief of staff”：每天委派上百个任务，并设置每天 9 点从 Gmail / Notion / Calendar 汇总日程和风险。这里真正被自动化的是注意力分配，而不是某个单点工具。
- 所以通用 Agent 的产品价值可以写成：减少信息搜索 + 减少协调 + 减少低价值操作 + 把重要异常及时抬到人面前。

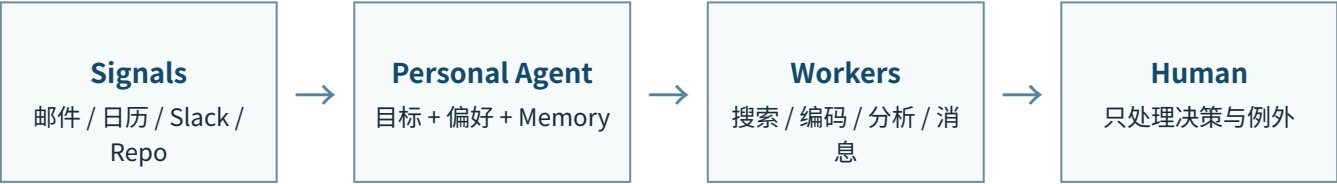
核心判断：这场是 Tibo 从 “Coding Agent” 转向 “通用个人 Agent” 最完整的一次公开解释：非代码任务已经成为主流使用，Agent 的下一步不是更多聊天能力，而是长期目标、后台执行、跨应用 context 和企业级 trust。

2. /goal 暗示了新的 Runtime 形态

- 普通 Chat 是 request/response；Task Agent 是 “给任务 -> 执行 -> 返回”；Goal Agent 则需要自己判断是否完成、何时重试、何时停下来求助。
- 当运行时间从 10 分钟变成数小时/数天，核心问题变成 durability：状态保存、预算、工具故障、环境变化、评测标准和恢复策略。
- Tibo 更进一步设想 24/7 Agent：它不必等明确 prompt 才工作，而是围绕长期目标持续发现和处理有价值事项。

SYSTEM VIEW

把访谈观点翻译成系统结构



这里最容易误解的一点

Agent 的“能力”并不等于裸模型能力。对真实任务而言，更接近：

$$\text{Agent Utility} = \text{Model} \times \text{Context} \times \text{Tools} \times \text{Runtime} \times \text{Safety} \times \text{Verification}$$

任何一项接近 0，端到端体验都会坍塌；这也是为什么模型 benchmark 提升并不总能等比例转化为产品收益。

工程上应该关注的闭环

Observe	任务 trace、tool call、权限、失败位置、人工介入、最终结果。
Diagnose	区分是模型、Harness、环境、数据、延迟还是安全策略导致失败。
Improve	把问题放回最合适的层修，而不是默认继续堆 prompt 或规则。
Verify	用可重复 eval 证明端到端成功率、成本和 latency 真正改善。

AI INFRA TAKEAWAYS

对 Agent Runtime / Serving 的启发

Durable execution	要从“模型会不会”升级成“任务能否跨小时恢复”：持久化状态、幂等 tool call、重试和 checkpoint。
Event-driven	24/7 Agent 不应该靠无脑轮询，需要 schedule、webhook、change detection 和优先级队列。
Context routing	不是把全部 Gmail/Slack 塞进上下文，而是检索、权限过滤、去重、时效性排序后再给模型。
Trust stack	Sandbox + 最小权限 + Auto-review + 审计 + 人类审批，共同决定企业能授权多大动作空间。
Success criteria	长目标必须可验证。没有 definition of done，算力只会被“持续思考”消耗。

给组内讨论的一个统一框架

把每项 Agent 优化问成四个问题：①端到端成功率提升多少；②wall-clock 缩短多少；③总算力/Token/工具成本增加多少；④风险与人工验收成本是否变高。单看模型分数、Kernel 峰值或单 Token 价格都不够。

$$T_{total} = T_{queue} + T_{prefill} + T_{decode} + T_{tool} + T_{network} + T_{environment} + T_{verification} + T_{human_wait}$$

WHAT TO BELIEVE / WHAT TO DO

需要打折看的部分

- “多数任务非代码” “每天 >100 个任务” 等是 Tibo 对内部/个人使用的描述，不代表所有 Codex 用户分布。
- /goal 和 24/7 Agent 中有明确未来方向成分，不能把当时的演示等同于所有用户已经可用。
- 官方 Forum 属于公司自有渠道，产品愿景表达天然更积极；本报告把愿景与已发生用例分开。

给组员的三个可执行动作

1. 设计长任务 benchmark：不仅测最终答案，还测 6h/24h 中断恢复、失败重试次数和总 token/工具成本。
2. 给 Agent 引入 priority + deadline + budget 三元组，避免“所有后台任务同权”。
3. 把企业安全评测改成“能力 x 授权范围 x 可检测性 x 可恢复性”的系统指标。

本场会谈的最终一句话

这场是 Tibo 从“Coding Agent”转向“通用个人 Agent”最完整的一次公开解释：非代码任务已经成为主流使用，Agent 的下一步不是更多聊天能力，而是长期目标、后台执行、跨应用 context 和企业级 trust。

原始资料

<https://forum.openai.com/public/videos/event-replay-codex-is-for-everyone-why-codex-matters-beyond-code-2026-05-13>

整理日期：2026-09-10。报告为技术阅读笔记，不代表 OpenAI 官方路线承诺；产品状态以最新官方文档为准。